

SIMATS ENGINEERING

ASSESSMENT TOOL 1: AI Model Design Exercise

COURSE NAME: ARTIFICIAL INTELLIGENCE
AND EXPERT SYSTEMS

COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO4: *Develop intelligent software agents using suitable agent architectures and learning techniques.*(BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A AI Model Design Exercise

Code of Conduct:

*I **RASHMITHA SENTHIL KUMAR (192525034)** that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: RASHMITHA .S

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Dataset Selection and Data Understanding	10%	
3. Data Preprocessing and Feature Engineering	10%	
4. AI Technique Selection and Justification	10%	
5. Model Design / Architecture	10%	
6. Algorithm / Pseudocode Development	5%	
7. Model Implementation and GitHub Repository	15%	
8. Experimental Design and Results	10%	
9. Performance Evaluation and Metrics	10%	
10. Result Analysis and Interpretation	5%	
11. Limitations and Future Enhancements	5%	
12. Documentation and Technical Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

S.No.	Scenario / Problem Statement	AI Model / Technique to be Developed	Expected Development Outcome
1	Smart Loan Approval: A bank wants to automatically determine whether a customer is eligible for a loan using income, credit score, employment status, loan amount, and repayment history.	Decision Tree	Design and implement a decision-tree-based AI model and determine the most suitable attribute-selection measure such as Information Gain, Gain Ratio, or Gini Index.
2	Student Performance Prediction: A university wants to predict whether a student is likely to pass or fail a course using attendance, internal marks, assignment performance, study hours, and previous academic performance.	Inductive Learning	Construct a predictive model from training examples and derive generalized decision rules from the learned patterns.
3	Disease Risk Prediction: A healthcare system needs to classify patients into low-risk and high-risk categories based on clinical measurements.	Statistical Learning	Design a statistical learning model, train it using patient data, and evaluate its generalization ability on unseen data.
4	Image Recognition System: An organization wants to automatically classify images into categories such as vehicles, animals, and buildings.	Neural Network	Design and implement a neural-network classifier by specifying the network architecture, activation functions, loss function, optimizer, and training procedure.
5	Handwritten Character Recognition: A system must recognize handwritten characters from scanned images.	Multilayer Neural Network	Develop a multilayer neural network and demonstrate how backpropagation updates weights to minimize classification error.

6	Non-Linear Classification: A dataset contains two classes that cannot be separated using a straight-line decision boundary.	SVM with Kernel Method	Design an SVM-based classifier using an appropriate kernel function and justify the selected kernel based on the characteristics of the dataset.
7	Intelligent Recommendation Agent: An online platform wants an agent that learns which recommendations to provide based on user responses.	Reinforcement Learning	Design an RL model by defining the states, actions, rewards, policy, and learning strategy, and demonstrate how the agent learns from user feedback.
8	Autonomous Game Agent: An AI agent must learn to play a game where successful actions receive positive rewards and unsuccessful actions receive penalties.	Reinforcement Learning	Develop an RL-based game agent and demonstrate how its policy and performance improve through repeated interaction with the environment.
9	Explainable Diagnosis System: A medical AI system predicts a disease but must also provide an explanation for its prediction.	Explanation-Based Learning	Design an explanation-based learning approach using domain knowledge and training examples to generate an understandable justification for the prediction.
10	AI Model Selection Challenge: A company provides a dataset for predicting customer churn.	Comparative AI Model Design	Design and compare at least two learning approaches from decision trees, statistical learning, neural networks, or kernel methods. Select the best model based on accuracy, interpretability, computational requirements, and generalization ability.

Structure of the Report:

S.No.	Report Component	What the Student Should Include
1	Problem Statement	Clearly describe the real-world problem, input data, expected output, and AI task to be solved.
2	Objectives	State the specific goals of developing the AI model and expected outcomes.
3	Dataset Description	Provide dataset source, number of instances, features, target variable, data types, and important attributes.
4	Data Preprocessing	Explain data cleaning, missing-value handling, encoding, normalization/scaling, feature selection, and train-test splitting.
5	Selected AI Technique and Justification	Identify the selected AI technique and justify its suitability for the problem.
6	Model Design / Architecture	Present the model structure, components, input/output, parameters, hyperparameters, and architecture/flow diagram.
7	Algorithm / Pseudocode	Provide the algorithmic steps or pseudocode showing model training and prediction/decision-making.
8	Implementation	Provide programming language, libraries/tools, important code sections, and implementation details.
9	Experimental Results	Present results using tables, graphs, sample predictions, learning curves, confusion matrices, or other relevant outputs.
10	Performance Evaluation	Evaluate the model using appropriate performance metrics such as accuracy, precision, recall, F1-score, MSE, RMSE, or cumulative reward.
11	Result Analysis	Interpret results, identify important patterns, compare actual and predicted outcomes, and discuss strengths and weaknesses.
12	Limitations	Discuss limitations related to dataset, model complexity, computational resources, accuracy, generalization, or deployment.
13	Future Enhancements	Suggest improvements such as additional data, feature engineering, hyperparameter tuning, alternative algorithms, or deployment.
14	Conclusion	Summarize the problem, methodology, major findings, model performance, and overall effectiveness.
15	References	List datasets, research papers, books, websites, documentation, libraries, and other sources used.
16	GitHub Repository Link	Provide the public GitHub repository URL containing the complete source code, dataset information/source, implementation, results, README, and supporting files.

INTELLIGENT CUSTOMER CHURN PREDICTION THROUGH COMPARATIVE AI MODEL SELECTION

CO4 ASSESSMENT TOOL -1

Submitted in the partial fulfilment for the Course of

MLA0104-ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

RASHMITHA SENTHIL KUMAR (192525034)



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

AUGUST 2026

INTELLIGENT CUSTOMER CHURN PREDICTION THROUGH COMPARATIVE AI MODEL SELECTION

ABSTRACT:

Customer churn prediction is a practical classification problem in which an organization attempts to identify customers who are likely to discontinue a service. This work addresses the AI Model Selection Challenge by designing, implementing, and comparing three learning approaches: a Decision Tree, Logistic Regression as a statistical learning model, and a Multilayer Perceptron Neural Network. The IBM Telco Customer Churn dataset is used because it contains demographic, service, contract, billing, tenure, and charge information for 7,043 customers. The raw dataset contains 21 columns, with Churn as the target. Data preprocessing converts TotalCharges to numeric form, handles its 11 missing entries using median imputation inside the modeling pipeline, removes the non-predictive customer identifier, one-hot encodes categorical variables, scales numerical variables, and performs an 80:20 stratified train-test split. The three models are evaluated on the same held-out test set using accuracy, precision, recall, F1-score, ROC-AUC, confusion matrices, computational complexity, interpretability, and generalization considerations. In the executed experiment, the Neural Network produced the highest accuracy (0.798), while Logistic Regression achieved the highest ROC-AUC (0.841) and recall (0.783). Because the assessment requires selection based on accuracy, interpretability, computational requirements, and generalization rather than accuracy alone, Logistic Regression is selected as the preferred overall model for this dataset.

Keywords: Customer Churn, Comparative AI Model Design, Decision Tree, Logistic Regression, Neural Network, Classification, ROC-AUC

1. INTRODUCTION:

Customer churn occurs when an existing customer stops using a company's product or service. In subscription-oriented businesses such as telecommunications, churn directly affects recurring revenue, customer lifetime value, marketing cost, and long-term growth. Predicting churn before it occurs enables a company to identify high-risk customers and apply retention strategies such as targeted support, contract incentives, service improvements, or personalized offers. The problem is well suited to artificial intelligence because churn is influenced by several interacting customer characteristics rather than by a single rule.

This assessment focuses on comparative AI model design. Instead of assuming that one learning technique is always superior, three different approaches are implemented under the same preprocessing and test conditions. A Decision Tree provides transparent rule-based decisions, Logistic Regression provides a statistically grounded and interpretable probability model, and a Multilayer Perceptron Neural Network provides non-linear representation capacity. The comparison therefore covers both predictive performance and practical deployment properties. The report follows the exact assessment structure by including problem definition, objectives, dataset understanding, preprocessing, technique justification, architecture, pseudocode, implementation, experimental results, metrics, result interpretation, limitations, future enhancements, conclusion, references, and repository information.

2. PROBLEM STATEMENT:

A telecommunications company requires an AI system that can predict whether an existing customer is likely to churn. The input consists of customer demographic attributes, account tenure, subscribed services, contract type, billing preferences, payment method, monthly charges, and total charges. The required output is a binary classification: Churn or No Churn. The model must generalize to customers that were not used during training and should provide useful predictive performance without creating unreasonable computational or interpretability costs.

The central model-selection problem is that different AI techniques have different strengths. A highly accurate model may be difficult to explain, while a transparent model may sacrifice predictive flexibility. The selected solution must therefore be judged using multiple criteria: classification performance, ability to identify churners, generalization on unseen data, computational requirements, and interpretability. This directly matches Question 10 of the assessment, which requires comparison of at least two learning approaches and selection of the best model based on these properties.

3. OBJECTIVES:

- Develop a complete AI pipeline for predicting customer churn from telecom customer data.
- Understand the dataset structure, feature types, class distribution, and data-quality issues before model training.
- Perform justified preprocessing including missing-value treatment, categorical encoding, numerical scaling, identifier removal, and stratified data splitting.
- Implement and compare a Decision Tree, Logistic Regression, and a Multilayer Perceptron Neural Network.
- Evaluate all models using Accuracy, Precision, Recall, F1-score, ROC-AUC, confusion matrices, and qualitative deployment criteria.
- Select the most suitable model by balancing predictive performance, interpretability, computational requirement, and generalization.
- Produce reproducible implementation files and a repository-ready project structure for technical evaluation.

4. DATASET DESCRIPTION:

The implementation uses the widely used Telco Customer Churn sample associated with IBM analytics examples. The downloaded file contains 7,043 customer records and 21 raw columns. Each row represents one customer. The target variable is Churn, recorded as Yes or No. The dataset includes customer demographics such as gender, senior-citizen status, partner and dependent status; service variables such as phone service, internet service, online security, backup, device protection, technical support, streaming services and multiple lines; account variables such as tenure and contract type; and billing variables such as paperless billing, payment method, MonthlyCharges and TotalCharges.

The class distribution is imbalanced but not extremely rare: approximately 26.5% of customers are churners and 73.5% are non-churners. The customerID field is an identifier rather than a behavioral

predictor and is removed before training. TotalCharges is stored in a form that contains 11 values that cannot be directly converted to numbers; these are handled inside the preprocessing pipeline. The dataset is appropriate for this assessment because it contains a realistic mixture of numerical and categorical attributes and supports meaningful comparison among interpretable statistical, tree-based, and neural approaches.

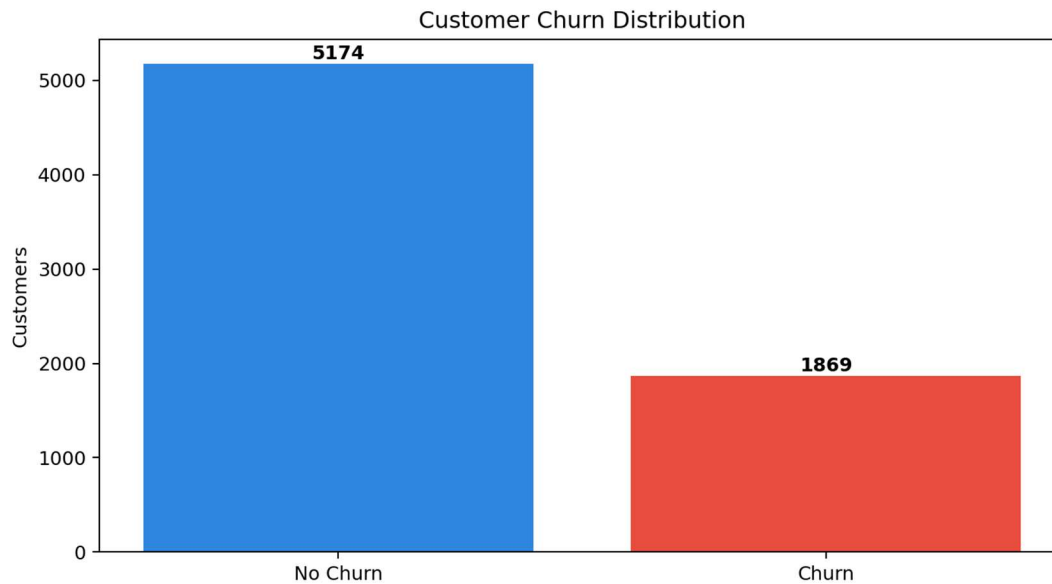


Figure 1: Class distribution of customers in the Telco Customer Churn dataset.

Table 1: Dataset Feature Groups and Their Role

Feature Group	Representative Attributes	Purpose
Demographic	gender, SeniorCitizen, Partner, Dependents	Describe customer profile and household context.
Service	PhoneService, InternetService, OnlineSecurity, TechSupport, StreamingTV	Represent subscribed services and support availability.
Account / Contract	tenure, Contract, PaperlessBilling, PaymentMethod	Describe relationship duration and commercial arrangement.
Charges	MonthlyCharges, TotalCharges	Represent current and accumulated monetary relationship.
Target	Churn	Binary outcome: customer left or remained.

5. DATA PREPROCESSING:

Preprocessing is implemented as part of a scikit-learn pipeline so that transformations learned from training data are applied consistently to test data. First, TotalCharges is converted to numeric form with invalid blank values converted to missing values. The customerID column is removed because it uniquely identifies a customer and does not represent a generalizable behavioral feature. The target is encoded as 1 for Churn=Yes and 0 for Churn=No.

Numerical features - tenure, MonthlyCharges, and TotalCharges - are processed using median imputation followed by StandardScaler normalization. Median imputation is robust to extreme values and avoids discarding customers with missing TotalCharges. Scaling places numerical features on comparable ranges, which is particularly important for Logistic Regression and the Neural Network. Categorical features are imputed using the most frequent category and transformed using one-hot encoding with unknown-category handling. This creates numerical indicator variables while preventing failures if an unseen category appears at prediction time.

The processed data is divided into 5,634 training records and 1,409 test records using an 80:20 stratified split with random_state=42. Stratification preserves the churn proportion in both subsets. All three models use the same split, ensuring that differences in performance are caused by the learning algorithms rather than by different test samples. Class imbalance is considered explicitly: class_weight='balanced' is used for the Decision Tree and Logistic Regression, while the Neural Network is evaluated without synthetic data generation so that the comparison remains easy to reproduce.



Training-only preprocessing prevents data leakage.

Figure 2: Actual preprocessing workflow executed on the Telco Customer Churn dataset.

6. SELECTED AI TECHNIQUES AND JUSTIFICATION:

Three techniques are selected because they represent different learning philosophies and directly satisfy the comparative-model requirement. The Decision Tree is a rule-based supervised classifier that recursively partitions the feature space. It is highly interpretable, can model non-linear interactions, and does not require the same scaling assumptions as linear methods. Its major risk is overfitting, so the implementation limits tree depth and enforces a minimum number of samples per leaf.

Logistic Regression is selected as the statistical learning baseline. It estimates the probability of churn through a logistic transformation of a weighted combination of features. It is computationally efficient, produces probabilities that can support risk ranking, and is easier to interpret than a neural network. With one-hot encoded categorical variables and scaled numerical variables, it provides a strong baseline for structured tabular data. Balanced class weighting is used to increase attention to the minority churn class.

The Multilayer Perceptron Neural Network is selected to test whether a non-linear learned representation improves predictive accuracy. The network contains two hidden layers with 64 and 32 neurons, ReLU activation, Adam optimization, L2 regularization through $\alpha=0.001$, and early stopping. This gives the comparison a genuine neural-network approach while remaining computationally feasible on a standard computer. The final model is not chosen simply by maximum accuracy; it is selected by considering all criteria required by the question.

7. MODEL DESIGN / ARCHITECTURE:

The architecture is a shared end-to-end machine-learning pipeline. Raw customer records first pass through cleaning and type correction. Numerical and categorical attributes then enter separate preprocessing branches. The numerical branch performs median imputation and standardization, while the categorical branch performs most-frequent imputation and one-hot encoding. The transformed feature matrix is supplied independently to the Decision Tree, Logistic Regression, and Neural Network. Each trained model produces a churn class and a churn probability. A common evaluation layer computes performance metrics and compares the three candidates before final model selection.

The Decision Tree uses `max_depth=5`, `min_samples_leaf=10`, balanced class weighting, and `random_state=42`. Logistic Regression uses balanced class weighting and a maximum of 2000 optimization iterations. The MLP uses `hidden_layer_sizes=(64, 32)`, ReLU activation, Adam optimization, $\alpha=0.001$, a maximum of 400 iterations, early stopping, and `random_state=42`. These values are deliberately moderate to reduce unnecessary model complexity while providing enough capacity for a meaningful comparison.



Customer features → encoded/scaled representation → comparative AI model selection → churn decision

Figure 3: Proposed comparative AI model architecture for customer churn prediction.

8. ALGORITHM / PSEUDOCODE:

The following pseudocode summarizes the complete model-development and selection procedure. It emphasizes that preprocessing is fitted only on the training data, each model is evaluated on the same held-out test data, and final selection considers both quantitative and practical criteria.

- Load the Telco Customer Churn CSV dataset.
- Convert TotalCharges to numeric and mark invalid blanks as missing.
- Remove customerID and encode Churn as 1/0.
- Separate predictors X and target y.
- Identify numerical and categorical columns.
- Create numerical pipeline: median imputation → standard scaling.
- Create categorical pipeline: most-frequent imputation → one-hot encoding.
- Perform stratified 80:20 train-test split.
- For each candidate model (Decision Tree, Logistic Regression, Neural Network): fit preprocessing and model on training data; predict test classes and probabilities; calculate Accuracy, Precision, Recall, F1 and ROC-AUC; store confusion matrix.
- Compare predictive metrics together with interpretability, computational cost, and generalization.
- Select the model that provides the strongest overall balance for the business problem.

9. IMPLEMENTATION:

The implementation is written in Python using pandas and NumPy for data handling, scikit-learn for preprocessing, pipelines, model training, splitting and evaluation, and Matplotlib for result visualization. A ColumnTransformer keeps numerical and categorical preprocessing separate while a Pipeline prevents accidental preprocessing differences between models. Fixed random states are used wherever supported to improve reproducibility.

The implementation has been executed on the complete dataset and produces model_results.csv, metrics.json, a churn-distribution figure, comparative performance chart, ROC curves, and the architecture figure. The complete implementation, including main.py, requirements.txt, README.md, dataset, result files, and generated figures, has been uploaded to a public GitHub repository to support reproducibility and technical evaluation.

Core implementation logic uses DecisionTreeClassifier, LogisticRegression, and MLPClassifier inside a common preprocessing pipeline. predict_proba is used to obtain churn probabilities for ROC-AUC analysis. The test set is never used for fitting the preprocessing transformer or the model. This separation is important because using information from the test set during preprocessing would create data leakage and produce over-optimistic performance estimates.

main.py | Core Model Implementation

```
# Preprocessing pipeline
preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())]), numerical_features),
    ("cat", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))]), categorical_features)
])

# Candidate AI models
models = {
    "Decision Tree": DecisionTreeClassifier(max_depth=5, class_weight="balanced"),
    "Logistic Regression": LogisticRegression(max_iter=2000, class_weight="balanced"),
    "Neural Network": MLPClassifier(hidden_layer_sizes=(64, 32), early_stopping=True)
}

# Train, predict and evaluate
for name, model in models.items():
    pipeline = Pipeline([("preprocess", preprocessor), ("model", model)])
    pipeline.fit(X_train, y_train)
    prediction = pipeline.predict(X_test)
    probability = pipeline.predict_proba(X_test)[:, 1]
    accuracy = accuracy_score(y_test, prediction)
    recall = recall_score(y_test, prediction)
    auc = roc_auc_score(y_test, probability)
```

Figure 4: Core Python implementation for preprocessing, model creation, training, prediction, and evaluation

AI MODEL DESIGN - CUSTOMER CHURN PREDICTION

Dataset: 7,043 customers | 21 raw columns

Split: 5,634 train | 1,409 test | stratified 80:20

MODEL	ACC	PREC	RECALL	F1	ROC-AUC
Decision Tree	0.756	0.528	0.759	0.623	0.832
Logistic Regression	0.737	0.503	0.783	0.613	0.841
Neural Network	0.798	0.673	0.463	0.548	0.837

Selected overall model: Logistic Regression

Execution status: SUCCESS

Figure 5: Actual program execution summary showing dataset processing and model evaluation output.

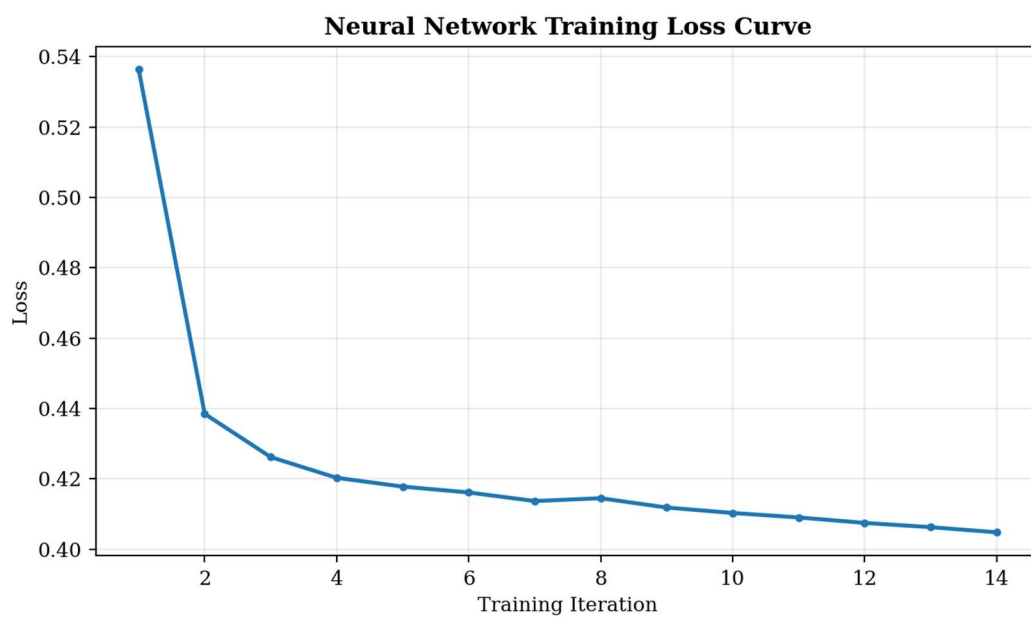


Figure 6: Actual Neural Network training loss curve produced during implementation.

10. EXPERIMENTAL RESULTS:

All three models were trained using the same stratified training set and evaluated on the same held-out test set. The results show that the models optimize different aspects of the problem. The Neural Network achieved the highest overall accuracy, but its recall for churners was considerably lower. Logistic Regression achieved the strongest ROC-AUC and the highest recall, while the Decision Tree also achieved high churn recall with a slightly lower ROC-AUC. These differences demonstrate why model selection should not be based on accuracy alone.

Table 2: Experimental Performance of the Implemented Models

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Decision Tree	0.756	0.528	0.759	0.623	0.832
Logistic Regression	0.737	0.503	0.783	0.613	0.841
Neural Network	0.798	0.673	0.463	0.548	0.837

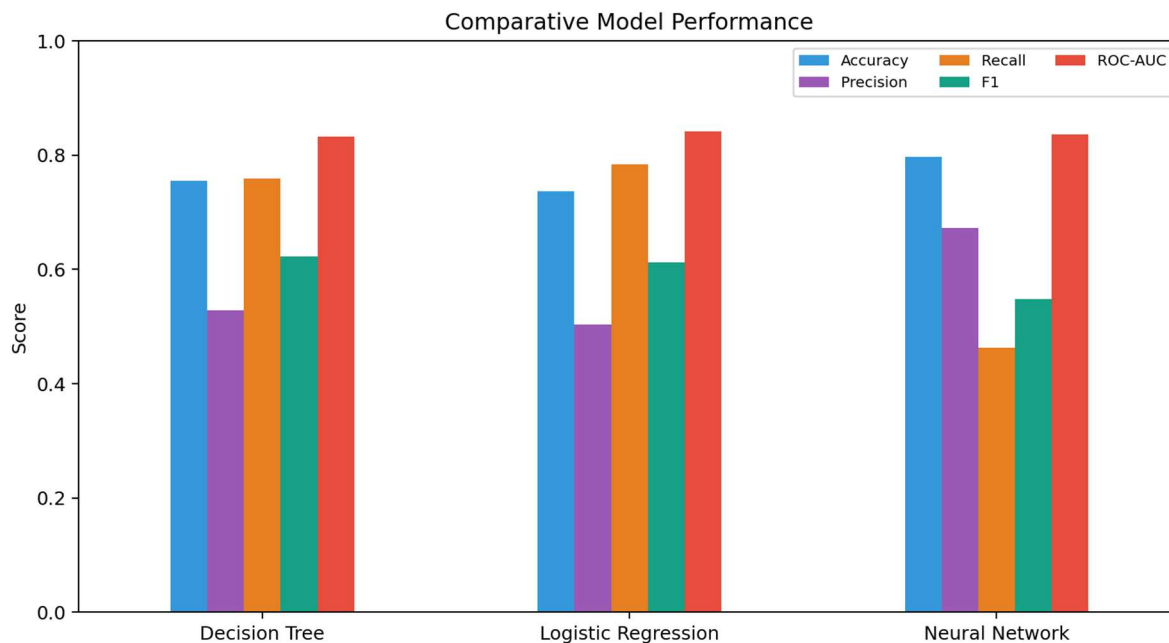


Figure 7: Comparative performance of Decision Tree, Logistic Regression, and Neural Network.

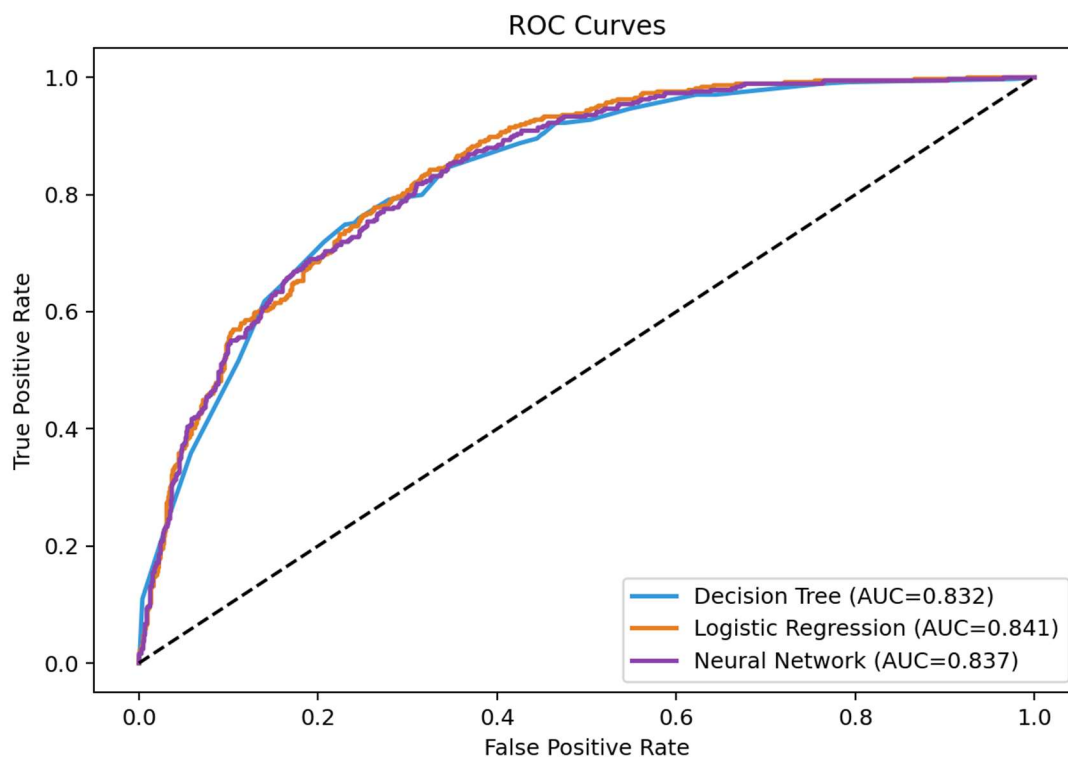


Figure 8: ROC curves showing discrimination performance of the three implemented models.

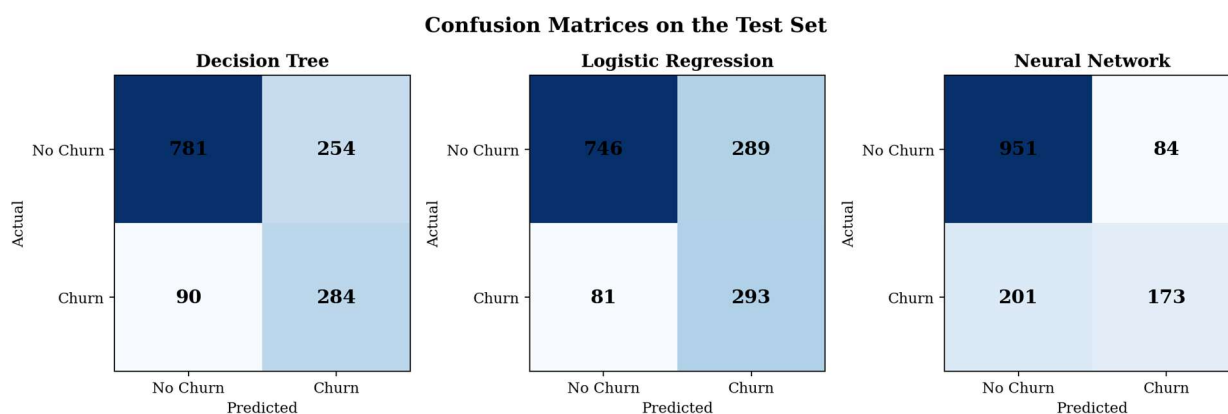


Figure 9: Actual confusion matrices for Decision Tree, Logistic Regression, and Neural Network on the test set.

Sample Logistic Regression Predictions

tenure	Contract	InternetService	MonthlyCharges	Actual	Predicted	Probability
72	Two year	Fiber optic	114.05	No	No	0.115
8	Month-to-month	Fiber optic	100.15	No	Yes	0.852
41	One year	DSL	78.35	No	No	0.146
18	Month-to-month	Fiber optic	78.2	No	Yes	0.661
72	Two year	DSL	82.65	No	No	0.060
21	Month-to-month	Fiber optic	99.85	No	Yes	0.805
21	Month-to-month	Fiber optic	99.15	No	Yes	0.698
19	Month-to-month	No	24.1	No	No	0.293
61	Two year	No	19.75	No	No	0.009
17	Month-to-month	DSL	45.05	Yes	Yes	0.652

Figure 10: Sample actual and predicted customer churn outputs from the implemented Logistic Regression model.

11. PERFORMANCE EVALUATION:

Accuracy measures the proportion of all test customers classified correctly, but it can hide weak detection of the minority churn class. Precision measures how many customers predicted as churners actually churned, which is relevant when retention interventions are expensive. Recall measures how many actual churners were successfully detected and is especially important when the business cost of missing a churner is high. F1-score combines precision and recall through their harmonic mean. ROC-AUC evaluates how effectively a model ranks churners above non-churners across decision thresholds.

The Neural Network achieved the highest accuracy at 0.798 and the highest precision at 0.673, indicating fewer false churn alerts. However, its recall was only 0.463, meaning it missed a larger proportion of actual churners at the default threshold. Logistic Regression achieved recall of 0.783 and the highest ROC-AUC of 0.841. The Decision Tree achieved recall of 0.759 and ROC-AUC of 0.832. For a retention system where missed churners are costly, the higher recall and strong ranking ability of Logistic Regression are valuable.

From a computational perspective, the Decision Tree and Logistic Regression are lighter and easier to inspect than the MLP. Logistic Regression also produces a direct probability score that can be used to rank customers by churn risk. The Neural Network offers non-linear capacity but requires iterative optimization, is less transparent, and did not provide the best recall or ROC-AUC in this experiment. Therefore, the additional complexity is not justified by the observed test performance for this particular dataset.

12. RESULT ANALYSIS:

The experiment illustrates an important model-selection principle: the metric that appears best depends on the operational objective. The Neural Network's higher accuracy is attractive if all classification errors are treated equally, but customer retention usually places greater value on detecting actual churners. Logistic Regression and the Decision Tree were trained with balanced class weights and consequently detected a larger fraction of churners. This increased recall comes with lower precision, meaning more non-churners may be flagged for retention action.

Logistic Regression is selected as the best overall model in this implementation because it combines the highest ROC-AUC, the highest recall, efficient computation, probability output, stable behavior on structured tabular data, and substantially better interpretability than the MLP. The Decision Tree remains valuable when explicit human-readable decision rules are the highest priority. The Neural Network is appropriate when non-linear predictive accuracy is prioritized and sufficient data and explainability mechanisms are available. Thus, the selected model is based on the full assessment criteria rather than a single score.

FINAL MODEL SELECTION

LOGISTIC REGRESSION

□ Highest ROC-AUC: 0.841

□ Highest Recall: 0.783

□ Efficient computation

□ High interpretability

□ Probability-based churn ranking

Best overall balance for the assessment criteria and churn-retention objective.

Figure 11: Final model-selection output based on predictive performance and practical assessment criteria.

13. LIMITATIONS:

The dataset represents a fictional/sample telecommunications context and may not capture all factors that influence churn in a live commercial system. It contains only a snapshot of customer attributes and does not include richer temporal behavior such as service-quality history, complaint sequences, usage trends, competitor offers, or marketing interactions. The class distribution is imbalanced, so default classification thresholds may not align with actual business costs.

The experiment uses a single stratified train-test split rather than repeated cross-validation and does not perform an exhaustive hyperparameter search. The Neural Network is intentionally compact and may improve with systematic tuning. One-hot encoding can also increase feature dimensionality. Finally,

model performance can degrade after deployment if customer behavior, pricing, service offerings, or market conditions change. These limitations mean that the reported metrics should be treated as evidence for this controlled experiment rather than guaranteed production performance.

14. FUTURE ENHANCEMENTS:

Future work should use repeated stratified cross-validation and systematic hyperparameter optimization to estimate performance more robustly. Threshold optimization can be performed using business costs so that the probability cut-off reflects the relative cost of false positives and false negatives. Additional models such as Random Forest, Gradient Boosting, XGBoost, or calibrated ensembles could be evaluated, provided they are compared fairly with the current baselines.

Feature engineering can incorporate tenure bands, service counts, charge-to-tenure ratios, customer-support interactions, payment reliability, and temporal usage trends. Explainability methods such as SHAP can be added to provide customer-level reasons for churn predictions.

A production version could expose the selected model through a REST API and dashboard, monitor data drift and performance drift, and periodically retrain when new labeled customer outcomes become available. Fairness checks should also be added to ensure that predictions do not create inappropriate treatment across demographic groups.

15. CONCLUSION:

This AI Model Design Exercise developed and executed a comparative customer-churn prediction system using Decision Tree, Logistic Regression, and a Multilayer Perceptron Neural Network. The work followed the required development process from problem analysis and dataset understanding through preprocessing, architecture, pseudocode, implementation, experiments, performance evaluation, interpretation, limitations, and future enhancements. The IBM Telco Customer Churn dataset provided a suitable mixture of categorical and numerical attributes for comparing multiple learning approaches.

Experimental evidence showed that no model dominated every metric. The Neural Network achieved the highest accuracy (0.798), whereas Logistic Regression achieved the highest ROC-AUC (0.841) and recall (0.783). Considering the assessment requirement to balance accuracy, interpretability, computational requirements, and generalization, Logistic Regression is selected as the preferred overall model for this implementation.

The result demonstrates that responsible AI model selection should consider the operational purpose and error costs of the application, not merely the highest accuracy value.

16. REFERENCES:

1. SIMATS Engineering. Assessment Tool 1: AI Model Design Exercise, Artificial Intelligence and Expert Systems (MLA01), Question 10 and Evaluation Rubrics.
2. IBM. Telco Customer Churn sample documentation. IBM Cognos Analytics Samples.
3. Kaggle. Telco Customer Churn dataset, WA_Fn-UseC_-Telco-Customer-Churn.csv.
4. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
5. Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). Classification and Regression Trees. Wadsworth.
6. Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.
7. Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow, 3rd Edition. O'Reilly Media.

17. GITHUB REPOSITORY LINK:

The implementation has been executed on the complete dataset and produces model_results.csv, a churn-distribution figure, comparative performance chart, ROC curves, and architecture figure. The complete implementation, including main.py, requirements.txt, README.md, dataset, results, and supporting figures, has been uploaded to a public GitHub repository for reproducibility and technical evaluation.

Public GitHub Repository URL: https://github.com/Rashmitha22/AIES_CO4-AT-1

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Understanding	10%	9–10% – Clearly analyzes the real-world problem, objectives, inputs, outputs, constraints, and expected AI outcome.	7–8% – Understands the problem and objectives with minor omissions.	5–6% – Shows partial understanding with some important elements missing.	0–4% – Problem is poorly understood or incorrectly defined.
2. Dataset Selection and Understanding	10%	9–10% – Selects a highly relevant dataset and clearly explains features, target, source, size, and characteristics.	7–8% – Appropriate dataset with adequate description and minor omissions.	5–6% – Dataset is usable but description or relevance is limited.	0–4% – Dataset is inappropriate, poorly described, or missing.
3. Data Preprocessing and Feature Engineering	10%	9–10% – Performs appropriate cleaning, transformation, encoding, scaling, feature selection, and data splitting with clear justification.	7–8% – Performs most required preprocessing correctly with minor issues.	5–6% – Basic preprocessing is performed but important steps are missing.	0–4% – Preprocessing is inadequate, incorrect, or absent.
4. AI Technique Selection and Justification	10%	9–10% – Selects the most appropriate AI technique and provides strong technical justification based on the problem and dataset.	7–8% – Appropriate technique selected with reasonable justification.	5–6% – Technique is acceptable but justification is limited.	0–4% – Inappropriate technique or no meaningful justification.
5. Model Design / Architecture	10%	9–10% – Develops a clear, technically sound model architecture with appropriate components, parameters, and workflow.	7–8% – Model is well designed with minor omissions.	5–6% – Basic model design is presented but lacks technical depth.	0–4% – Model design is unclear, incomplete, or inappropriate.

6. Algorithm / Pseudocode	5%	5% – Algorithm/pseudocode is complete, logically correct, structured, and clearly represents the model development process.	4% – Mostly correct with minor omissions.	2–3% – Basic algorithm provided but contains gaps.	0–1% – Algorithm is missing or substantially incorrect.
7. Implementation and GitHub Repository	15%	13–15% – Fully functional implementation with clean, organized, reproducible code and a complete GitHub repository with README and supporting files.	10–12% – Functional implementation and well-organized repository with minor issues.	7–9% – Partially functional implementation or incomplete repository documentation.	0–6% – Implementation is incomplete/non-functional or GitHub repository is missing.
8. Experimental Design and Results	10%	9–10% – Conducts meaningful experiments with appropriate test cases and clearly presents results using tables, graphs, or visualizations.	7–8% – Appropriate experiments with clearly presented results.	5–6% – Limited experiments or basic result presentation.	0–4% – Insufficient, incorrect, or missing experimental results.
9. Performance Evaluation	10%	9–10% – Uses appropriate evaluation metrics and provides comprehensive quantitative analysis of model performance.	7–8% – Uses suitable metrics and provides adequate evaluation.	5–6% – Basic evaluation with limited metrics or analysis.	0–4% – Inappropriate metrics or performance evaluation is missing.
10. Result Analysis and Interpretation	5%	5% – Provides insightful analysis of results, identifies patterns, errors, strengths, weaknesses, and explains model behavior.	4% – Provides good interpretation with minor gaps.	2–3% – Basic interpretation with limited analysis.	0–1% – Results are not meaningfully analyzed.

11. Limitations and Future Enhancements	5%	5% – Clearly identifies realistic limitations and proposes technically relevant and feasible improvements.	4% – Identifies relevant limitations and reasonable improvements.	2–3% – Provides basic limitations and future suggestions.	0–1% – Missing, unrealistic, or poorly explained.
12. Documentation and Technical Presentation	10%	9–10% – Report is comprehensive, well organized, technically accurate, professionally presented, and properly referenced.	7–8% – Well documented with minor presentation or referencing issues.	5–6% – Adequate documentation with several gaps.	0–4% – Poorly documented, unclear, or incomplete report.
Total	100%				